TALLER PRÁCTICO – BLOQUE 5

De clases aisladas a objetos que colaboran

Interfaces · Asociación · Agregación · Composición · Herencia · Componentes UML

Dato	Información
Caso conductor	SmartLibrary
Duración	120 minutos
Modalidad	Parejas o equipos de 3 estudiantes
Herramientas	Draw.io / Visual Paradigm + Java (IDE de preferencia)
Producto	Diagrama UML actualizado + implementación Java parcial + justificación de decisiones
Medio de entrega	repositorio de git

1. Propósito del taller

Evolucionar el diseño construido en el Bloque 4 para que los objetos de SmartLibrary dejen de verse como elementos aislados y comiencen a expresar relaciones estructurales y contratos de colaboración. El objetivo no es utilizar todos los tipos de relación posibles, sino seleccionar cada relación porque representa correctamente una regla o significado del problema.

Al finalizar, el estudiante deberá poder diferenciar asociación, agregación y composición; justificar cuándo una generalización mediante herencia tiene sentido; utilizar una interfaz como contrato de comportamiento; y representar una vista funcional introductoria mediante componentes UML.

2. Punto de partida: SmartLibrary

SmartLibrary administra estudiantes, ejemplares, libros, préstamos y reservas. Para este bloque se incorporan requisitos adicionales que obligan a revisar el diseño:

- R10. Un libro puede tener varios ejemplares físicos. Cada ejemplar corresponde a un único libro.
- R11. Un préstamo registra las renovaciones realizadas. Cada renovación conserva la fecha anterior de devolución, la nueva fecha y la fecha en que se realizó la renovación.
- R12. El sistema maneja usuarios de biblioteca. Los estudiantes y bibliotecarios comparten identificación, nombre y correo, pero poseen información particular.
- R13. Algunos usuarios pueden recibir notificaciones del sistema.
- R14. Para comprender funcionalmente la solución se desea organizar el diseño en Gestión de Usuarios, Gestión de Catálogo, Gestión de Préstamos y Gestión de Reservas.

Regla de trabajo: no asuma que toda relación “tiene-un” es composición ni que compartir atributos justifica herencia. Cada decisión debe sustentarse.

3. Resultados de aprendizaje

- Representar asociaciones UML coherentes con las necesidades de colaboración entre objetos.
- Distinguir agregación y composición a partir de la independencia y ciclo de vida de las partes.
- Evaluar críticamente si una relación de herencia expresa realmente una relación «es-un».
- Definir e implementar una interfaz Java sencilla como contrato de comportamiento.
- Construir una vista funcional introductoria mediante componentes UML sin convertirla en un diseño de arquitectura.
- Justificar las decisiones tomadas y mantener trazabilidad con los requisitos.

4. Distribución del tiempo

Actividad	Tiempo
1. Diagnóstico de relaciones	15 min
2. Asociación, agregación y composición	25 min
3. Herencia: ¿es-un o solo se parecen?	20 min
4. Interfaces y contrato de notificación	20 min
5. Implementación representativa en Java	25 min
6. Vista funcional mediante componentes UML	10 min
Cierre y socialización	5 min

Actividad 1 – Diagnóstico de relaciones

Analicen las siguientes parejas. Antes de dibujar símbolos UML, describan en lenguaje natural qué significa la relación y si los objetos pueden existir independientemente.

Elementos	¿Qué relación existe en el problema?	¿Existencia independiente?	Decisión inicial
Estudiante – Préstamo	Asociación	si	Asociación
Libro – Ejemplar	Agregación	si	Agregación
Préstamo – Renovación	Composición	no	Composición

Usuario – Estudiante	Generalización/herencia	Un estudiante puede ser tratado como un Usuario. La clase Usuario contiene los datos comunes: • identificación • nombre • correo Mientras que Estudiante contiene: • código estudiantil • programa académico.	Herencia
Usuario – Bibliotecario	Generalización/herencia	Bibliotecario tiene: Código de empleado y turno	Herencia

Preguntas de discusión:

1. ¿Qué dos clases se relacionen significa necesariamente que una sea parte de la otra?

Rta No. Dos clases pueden relacionarse porque necesitan conocerse o colaborar. Por ejemplo, Estudiante y Préstamo están relacionados, pero el estudiante no es una parte del préstamo ni el préstamo es una parte del estudiante.

2. ¿Qué evidencia del requisito utilizaron para decidir?

Se utilizaron principalmente los requisitos R10, R11, R12 y R13.

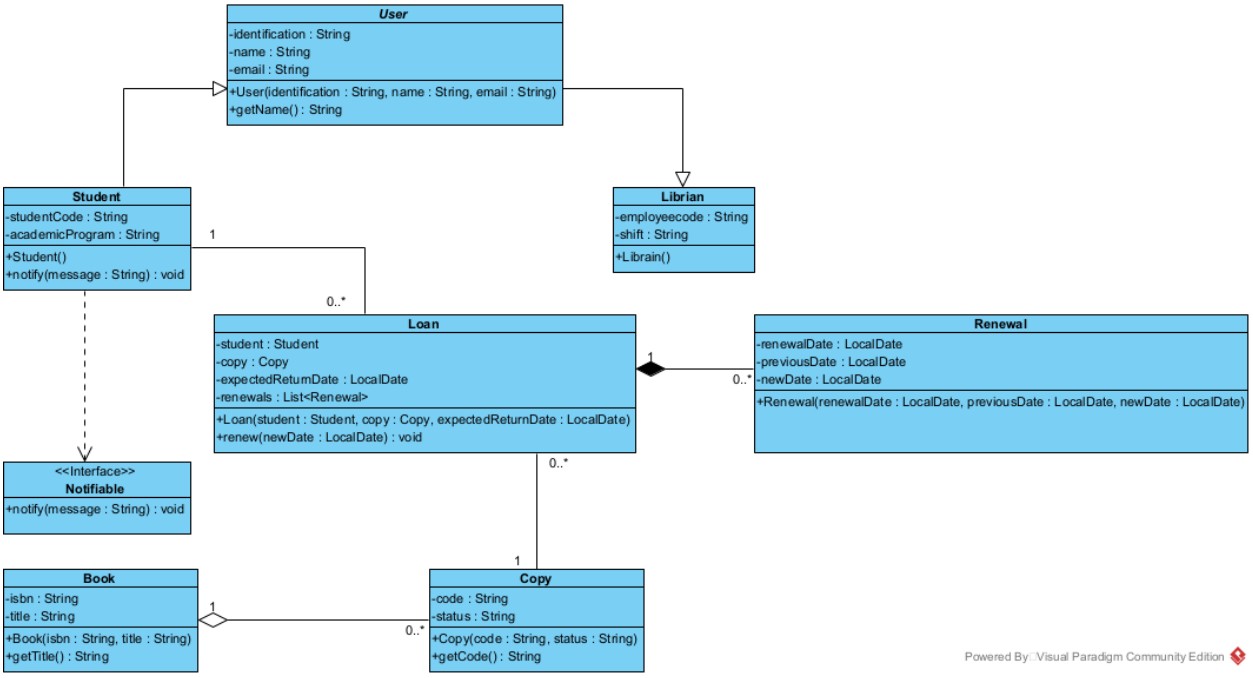
- R10 permite analizar Libro–Ejemplar.
- R11 permite determinar que Renovación pertenece al historial de un Préstamo.
- R12 permite analizar la generalización entre Usuario, Estudiante y Bibliotecario.
- R13 permite determinar qué objetos pueden cumplir el contrato de notificación

3. ¿Cuál de las relaciones anteriores sería incorrecto decidir únicamente mirando los nombres de las clases?

Libro – Ejemplar y **Préstamo – Renovación**. Solo por sus nombres podríamos pensar que ambas son relaciones "tiene", pero eso no permite determinar automáticamente si son asociación, agregación o composición.

Actividad 2 – Asociación, agregación y composición

Construyan un fragmento de diagrama de clases que incluya Libro, Ejemplar, Estudiante, Préstamo y Renovación. Incluyan multiplicidades y seleccionen el tipo de relación que consideren correcto.



Para cada relación completen:

Relación	**Tipo seleccionado**	**Justificación basada en el dominio**
Estudiante – Prestamo	Asociación	El estudiante y el préstamo tienen existencia conceptual independiente.
Prestamo – Ejemplar	Asociación	El préstamo utiliza un ejemplar, pero no controla su ciclo de vida.
Libro – Ejemplar	Agregación	El libro agrupa ejemplares, pero se adopta el supuesto de que el

		ejemplar puede continuar existiendo independientemente.
Prestamo – Renovacion	Composición	La renovación pertenece al historial del préstamo y no tiene sentido independiente.

Utilicen estas preguntas como criterio, no como fórmula automática:

- ¿Solo necesito expresar que los objetos se conocen o colaboran?
- ¿Existe una relación conceptual todo–parte?
- ¿La parte tiene sentido y ciclo de vida independiente del todo?
- ¿La eliminación conceptual del todo implica que la parte deja de tener significado?

Reto de razonamiento: Dos equipos pueden proponer relaciones distintas si el enunciado deja una regla abierta. En ese caso, documenten el supuesto que hace válida su decisión.

Actividad 3 – Herencia: ¿es-un o solo se parecen?

Considere la siguiente información:

Usuario

- identificacion
- nombre
- correo

Estudiante

- codigoEstudiantil

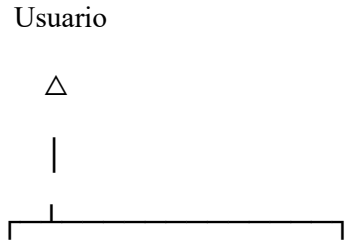
- programaAcademico

Bibliotecario

- codigoEmpleado

- turno

Evalúen la propuesta:



4. ¿Estudiante es realmente un tipo de Usuario dentro de SmartLibrary? Justifique.

Sí. Dentro de SmartLibrary, un estudiante es un usuario de la biblioteca que además posee información específica como código estudiantil y programa académico. Por lo tanto,
`Estudiante es un Usuario`

5. ¿Bibliotecario es realmente un tipo de Usuario? Justifique.

Si, el bibliotecario tambien representa un usuario del sistema y comparte los atributos como:

- Identificación
- Nombre
- Correo
- Código de empleado
- Turno

Por esto se concluye que Bibliotecario es un usuario

6. ¿La existencia de identificación, nombre y correo repetidos sería suficiente para crear la superclase Usuario? ¿Por qué?

No, la repetición de atributos no es suficiente, la herencia debe representar una relación de “es un”, en este caso si es valido porque tanto como estudiante y bibliotecario representan tipos de usuarios dentro del sistema.

7. Propongan al menos una situación en la que crear una superclase únicamente para evitar atributos repetidos produciría un diseño conceptualmente débil.

Por ejemplo, supongamos que `Libro` y `Estudiante` tuvieran un atributo llamado `nombre`.

No tendría sentido crear:

```
NombreComun
    △
    ├── Libro
    └── Estudiante
```

Solo para evitar repetir el atributo `nombre`.

Aunque ambos tengan un atributo con el mismo nombre, no significa que conceptualmente sean tipos de la misma entidad. La relación tendría que representar un significado real del dominio.

Si consideran válida la generalización, actualicen el diagrama. Si no, propongan otra solución y justifíquenla.

Actividad 4 – Interfaces: expresar un contrato

SmartLibrary necesita que determinados objetos puedan recibir notificaciones. En este bloque no diseñaremos todavía patrones ni principios SOLID. Solo queremos expresar un contrato de comportamiento.

```
public interface Notificable {
  void notificar(String mensaje);
}
```

Realicen lo siguiente:

8. Decidan qué clase o clases de su diseño deberían implementar Notificable y justifiquen la decisión.

Se propone que Estudiante implemente Notificable. Esto se debe a que el sistema puede necesitar comunicar al estudiante información relacionada con sus préstamos, por ejemplo: "Su préstamo fue renovado."

9. Representen la interfaz y la relación de realización en UML.

10. Explique qué garantiza el contrato Notificable y qué NO especifica.

Garantiza que cualquier clase que implemente Notificable debe proporcionar el metodo: void notificar(string mensaje);

No especifica si la notificación se realiza mediante correo electrónico, mensaje interno, notificación push u otro mecanismo, la interfaz establece que operación debe ofrecer la clase, pero no como realizarla.

11. Explique por qué una interfaz no equivale a una superclase.

Porque esta representa principalmente un contrato de comportamiento. Una super clase representa una relación de generalización entre clases y puede proporcionar atributos y comportamiento común.

Complete:

Pregunta	**Respuesta del equipo**
¿Qué promete una clase que implementa Notificable?	Promete implementar el método: Void notificar(String mensaje);
¿La interfaz define cómo se envía el mensaje?	No, solo define la operación que debe estar disponible. La forma concreta de realizar la

	notificación queda a cargo de la clase que implementa la interfaz.
¿Puede una clase heredar de Usuario e implementar Notificable al mismo tiempo? Explique.	Si, un ejmplo es el siguiente: Public class Estudiante extends Usuario implements Notificable Esto permite que Estudiante herede los atributos y comportamiento de usuario y, adicionalmente, cumpla el contrato de Notificable.

Actividad 5 – Implementación representativa en Java

Implemente únicamente el fragmento necesario para validar las decisiones de diseño. No construya una aplicación completa.

5.1 Usuario y especializaciones

```
public abstract class Usuario {
  private String identificacion;
  private String nombre;
  private String correo;

  public Usuario(String identificacion, String nombre, String correo) {
    this.identificacion = identificacion;
    this.nombre = nombre;
    this.correo = correo;
  }
```

```
    public String getNombre() {
        return nombre;
    }
}
```

```
        public class Estudiante extends Usuario implements Notificable {
    private String codigoEstudiantil;
    private String programaAcademico;

    // TODO: constructor

    @Override
    public void notificar(String mensaje) {
        // TODO: simular la notificación en consola
    }
}
```

Implemente también Bibliotecario como especialización de Usuario. Decida justificadamente si debe implementar Notificable.

5.2 Préstamo y renovaciones

```
        public class Renovacion {
    private LocalDate fechaRenovacion;
    private LocalDate fechaAnterior;
    private LocalDate nuevaFecha;
```

```
    // TODO: constructor y getters necesarios
}
```

```
        public class Prestamo {
    private Estudiante estudiante;
    private Ejemplar ejemplar;
    private LocalDate fechaPrevistaDevolucion;
    private List<Renovacion> renovaciones = new ArrayList<>();

    public void renovar(LocalDate nuevaFecha) {
        // TODO:
        // 1. validar que nuevaFecha sea posterior a la fecha actual prevista
        // 2. crear la Renovacion
        // 3. almacenarla
        // 4. actualizar fechaPrevistaDevolucion
    }
}
```

La implementación debe impedir una renovación cuya nueva fecha sea igual o anterior a la fecha prevista vigente.

5.3 Prueba mínima

```
        // Ejemplo orientador
Estudiante estudiante = ...;
Ejemplar ejemplar = ...;
Prestamo prestamo = ...;
```

```
prestamo.renovar(LocalDate.of(2026, 10, 15));
estudiante.notificar("Su préstamo fue renovado.");

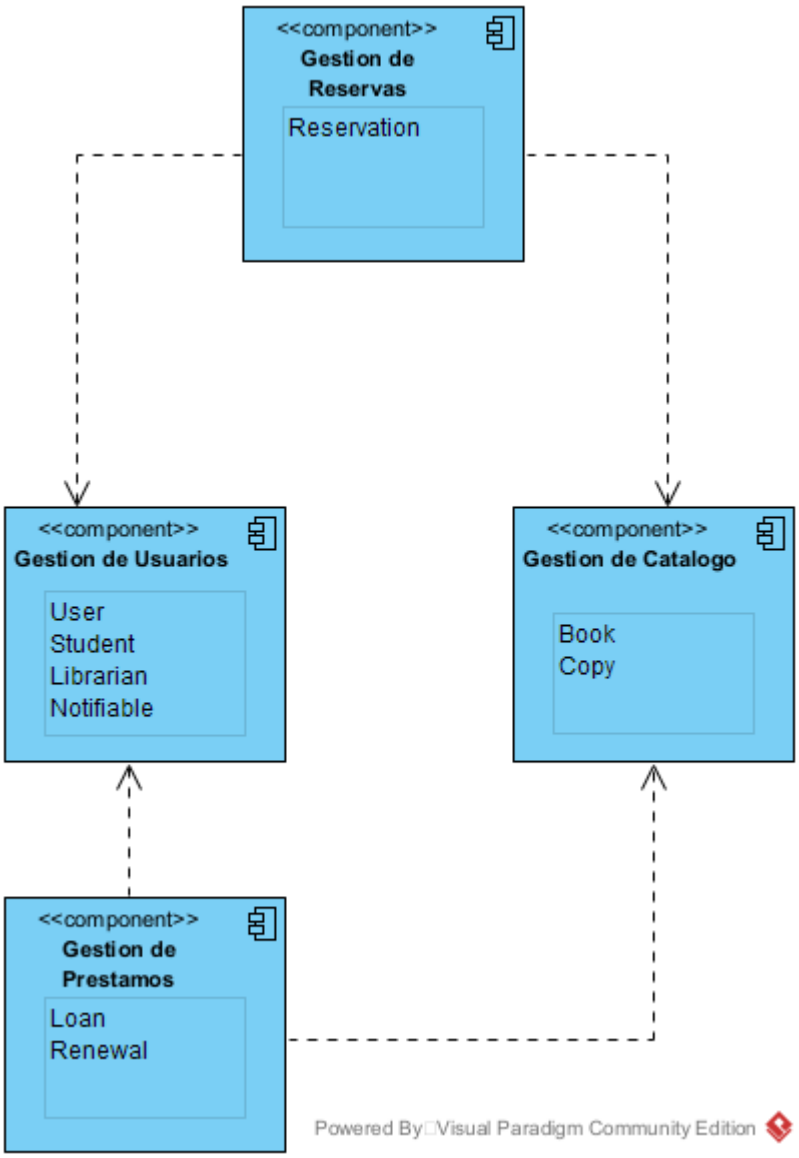
System.out.println(...); // mostrar la nueva fecha
System.out.println(...); // mostrar cantidad de renovaciones
```

Agregue una segunda prueba que intente registrar una renovación inválida y documente qué debería ocurrir.

Actividad 6 – Vista funcional mediante componentes UML

Construyan una vista funcional introductoria de SmartLibrary utilizando cuatro componentes UML:

- Gestión de Usuarios
- Gestión de Catálogo
- Gestión de Préstamos
- Gestión de Reservas



Para cada componente identifique qué clases de su diseño ubicaría principalmente allí y qué colaboración funcional necesita con los demás.

Componente	Clases relacionadas	¿De qué otro componente necesita información? ¿Para qué?

Gestión de Usuarios	Usuario, Estudiante, Bibliotecario, Notificable	Necesita información de gestión de prestamos para conocer los prestamos de los estudiantes y enviar notificaciones relacionadas con ellos.
Gestión de Catálogo	Libro, Ejemplar	Necesita información de gestión de prestamos para conocer el estado de los ejemplares y de gestión de reservas para conocer las reservas realizadas sobre los libros.
Gestión de Préstamos	Préstamo, Renovación	Necesita información de gestión de usuarios para identificar al estudiante que realiza el préstamo y de gestión de catalogo para identificar el ejemplar que se presta.
Gestión de Reservas	Reserva	Necesita información de gestión de usuarios para identificar al estudiante que realiza la reserva y de gestión de catalogo para identifica el libro reservado.

Dibuje el diagrama de componentes. En este bloque la vista es funcional y de alto nivel: no diseñe microservicios, infraestructura, contenedores, despliegue, APIs ni bases de datos.

8. Conclusión

El desarrollo del taller permitió comprender cómo las clases de SmartLibrary pueden relacionarse para representar de mejor manera el funcionamiento del sistema. Se identificaron diferentes tipos de relaciones UML de acuerdo con el significado de cada elemento dentro del dominio. La asociación se utilizó para representar la colaboración entre estudiantes, préstamos y ejemplares, mientras que la agregación permitió representar la relación entre libros y ejemplares. La composición se aplicó entre préstamos y renovaciones debido a la dependencia de estas últimas respecto al préstamo. También se utilizó la herencia para representar a estudiantes y bibliotecarios como tipos de usuarios y una interfaz para establecer el contrato de notificación. Finalmente, el diagrama de componentes permitió organizar las funcionalidades principales de SmartLibrary en módulos de usuarios, catálogo, préstamos y reservas. La implementación en Java permitió comprobar estas decisiones mediante una renovación válida y otra inválida.

8. Criterios de evaluación

Criterio	Peso
Uso y justificación de asociación, agregación y composición	25 %
Herencia correctamente justificada	15 %
Interfaz y contrato de comportamiento	15 %

Coherencia del diagrama de clases y multiplicidades	15 %
Implementación Java representativa y funcional	15 %
Vista funcional mediante componentes UML	10 %
Argumentación técnica y claridad	5 %
TOTAL	100 %

9. Lista de verificación del estudiante

- □ No utilicé composición solo porque una clase contiene otra.
- □ Justifiqué las relaciones a partir del significado del dominio.
- □ La herencia expresa «es-un» y no solamente reutilización de atributos.
- □ Puedo explicar la diferencia entre heredar una clase e implementar una interfaz.
- □ Mis multiplicidades son coherentes con los requisitos.
- □ El código refleja las relaciones mostradas en UML.
- □ Prestamo protege la regla de renovación.
- □ Mi diagrama de componentes muestra agrupaciones funcionales, no una arquitectura de microservicios.
- □ No adelanté GRASP, SOLID ni patrones de diseño.

Guía breve para el profesor

Esta sección no debe entregarse como solución cerrada antes del taller. Su propósito es orientar la discusión y la retroalimentación.

Decisiones esperadas y matices

- Estudiante–Prestamo y Prestamo–Ejemplar pueden trabajarse naturalmente como asociaciones: los objetos relacionados tienen existencia conceptual independiente.
- Prestamo–Renovacion es un buen caso de composición si Renovacion se define exclusivamente como parte del historial interno de un préstamo y carece de sentido independiente.
- Libro–Ejemplar puede generar discusión. Es válido exigir que el estudiante explicite el supuesto de ciclo de vida. No premiar el símbolo por sí solo; evaluar la justificación.
- Usuario–Estudiante/Bibliotecario puede modelarse como generalización si Usuario representa una abstracción real del sistema y las especializaciones satisfacen la relación «es-un». La mera repetición de atributos no basta.
- Notificable debe evaluarse como contrato de capacidad. Evitar introducir todavía inversión de dependencias, Strategy u otros patrones.
- La vista de componentes es introductoria y funcional. No convertir la actividad en arquitectura de software.

Preguntas de retroalimentación

- ¿Qué regla del problema demuestra que esta parte no puede existir independientemente?
- Si mañana cambia el ciclo de vida de la parte, ¿seguiría siendo composición?
- ¿Podría reemplazar un Usuario por un Estudiante en este contexto sin romper el significado?
- ¿Qué promete exactamente Notificable a quien utiliza un objeto que cumple ese contrato?
- ¿Qué diferencia existe entre una clase del diagrama y un componente de la vista funcional?